

Gala Google OAuth Migration and GCP Project Recovery Options

Executive Summary

Gala production is currently running at <https://www.learnkala.com> on Heroku. That deployment is tied to a Google OAuth configuration in an orphaned Google Cloud project. The project is associated with a disabled **umich.edu** account, which limits our ability to manage OAuth redirect URIs, rotate credentials, or support new environments.

An AWS migration candidate is available in [galahq/gala#787](#). The main decision is whether to recover administrative control of the existing Google Cloud project, move OAuth to a new managed project, or combine both paths to reduce production risk.

Current State

Area	Current Status	Risk
Production app	Heroku deployment at https://www.learnkala.com	Working today, but dependent on an unmanaged OAuth project.
Google OAuth project	Project number 1047090780817	Owned by a disabled umich.edu account.
OAuth redirect management	Limited or unavailable	New AWS or .dev callback URLs may not be addable in the existing project.
AWS migration	Candidate available in galahq/gala#787	Needs OAuth continuity before production cutover.
User identity	Google OAuth uid values are project-specific	Users can be duplicated if a new OAuth project is used without migration logic.

Decision Options

Option	Description	Pros	Cons	Recommended Use
1. Recover the orphaned GCP project	Use Google domain or admin recovery to regain control of the existing project.	Preserves existing OAuth client and user uid values. Avoids user identity migration.	Google recovery may take time and is not guaranteed. Production remains dependent on a project we do not currently control.	Best long-term outcome if recovery is feasible.
2. Create a new	Create new Google OAuth credentials	Puts credentials under known	Existing Google uid values will not	Best fallback if recovery is delayed

Option	Description	Pros	Cons	Recommended Use
managed OAuth project	under a controlled Gala-owned project and update environments to use them.	ownership. Supports AWS and <code>.dev</code> callback URLs immediately.	match. Requires match-on-email migration logic to prevent duplicate users.	or denied.
3. Hybrid path	Start recovery for the orphaned project while preparing the new managed OAuth project and migration patch.	Keeps the low-risk recovery path open while creating an actionable fallback.	Requires coordinating both workstreams.	Recommended practical path.
4. Delay AWS cutover	Keep Heroku production unchanged until project recovery is complete.	Lowest immediate production change.	Blocks or delays AWS migration and leaves credentials unmanaged.	Use only if stakeholder risk tolerance favors no OAuth changes before recovery.

Option 1: Orphaned Project Recovery Plan

Context: The production Google Cloud project, project number `1047090780817`, is owned by a disabled `umich.edu` account.

Objective: Regain access so Gala can manage OAuth settings, add new redirect URIs, and rotate credentials safely.

Step	Action	Resource or Owner	Notes
1	Initiate domain or admin recovery	Google Admin recovery process	Start with the Google recovery form for the domain or organization account tied to the project.
2	Prove domain ownership	Gala DNS administrator	Add the required CNAME or TXT record to <code>learngala.com</code> DNS.
3	Request escalation	Google Support	Ask Google to promote a Gala-controlled admin account to an administrator or project owner role.
4	Validate access	Gala engineering	Confirm OAuth client visibility, redirect URI management, and secret rotation access.
5	Add AWS and <code>.dev</code> redirects	Gala engineering	Add callback URLs needed by the AWS candidate and staging environments.

Option 2: New Managed OAuth Project

Google OAuth user IDs are project-specific. If Gala moves OAuth to a new Google Cloud project, existing users may appear as new users unless the application links accounts by verified email during the migration window.

Identity Migration Patch

Implement a match-on-email fallback when the Google OAuth `uid` is not found. This keeps existing user accounts attached to the new OAuth project identity instead of creating duplicates.

```
# app/models/user.rb
def self.from_omniauth(auth)
  user = User.find_by(provider: auth.provider, uid: auth.uid)

  if user.nil?
    user = User.find_by(email: auth.info.email)

    if user
      user.update!(
        provider: auth.provider,
        uid: auth.uid
      )
    else
      user = User.create!(
        email: auth.info.email,
        provider: auth.provider,
        uid: auth.uid,
        password: Devise.friendly_token[0, 20]
      )
    end
  end

  user
end
```

New OAuth Client Setup

Run the following command in Google Cloud Shell to create credentials for the new testing and production-development environments in project `project-78c8a048-9020-484c-b18`.

```
gcloud iam oauth-clients create gala-migration-client \
  --project=project-78c8a048-9020-484c-b18 \
  --location=global \
  --client-type=confidential-client \
  --allowed-grant-types=authorization-code-grant,refresh-token-grant \
  --allowed-scopes=openid,email,profile \
  --allowed-redirect-uris="https://learngala.dev/users/auth/google_oauth2/callback,https://dev.learngala.dev/users/auth/google_oauth2/callback"
```

Option 3: Hybrid Recommendation

The recommended approach is to run the recovery path and the fallback migration path in parallel:

Workstream	Action	Exit Criteria
Recovery	Attempt to regain control of project 1047090780817 .	Gala-owned admin account can manage OAuth client settings and secrets.
Migration	Prepare a new managed OAuth project and match-on-email fallback.	AWS candidate can authenticate users without duplicate account creation.
Cutover	Use whichever OAuth path is ready and lower risk at launch time.	Production users can sign in successfully after AWS cutover.

Safe Key Rotation Plan

To rotate OAuth secrets without kicking current users out:

Step	Action	Notes
1	Generate a secondary client secret	In the GCP Console, create a second client secret for the same OAuth client ID.
2	Deploy the new secret to AWS	Update AWS environment variables for the migration candidate.
3	Keep Heroku active	Leave the old Heroku secret in place during migration. Google will honor both secrets simultaneously.
4	Cut over DNS	Move production traffic to AWS after authentication has been verified.
5	Decommission the old secret	Delete the old secret only after DNS has fully cut over and production sign-in is verified.

Stakeholder Decision Needed

Decision	Options	Recommended Choice
Primary path	Recover existing project, create new project, or run both paths	Run both paths until one is confirmed production-ready.
AWS cutover timing	Wait for recovery or proceed with fallback migration	Proceed only after OAuth sign-in is verified against the chosen path.
Identity migration tolerance	Allow match-on-email migration or require original Google uid continuity	Allow match-on-email as a controlled fallback if recovery is not available.

Prepared By

Google Cloud Infrastructure Architect
Atomic Object